

System Level I/O

元培数科 王恩博

Let's get started →

2025/11/26



Knowledge Review

Unix I/O

Unix I/O

- 所有的 I/O 设备都被模型化为 **文件**
- 输入输出 **等价于文件的读写**
- 研讨题1.1

- **Cool fact: All I/O devices are represented as files:**

- `/dev/sda2` (/usr disk partition)
- `/dev/tty2` (terminal)

- **Even the kernel is represented as a file:**

- `/boot/vmlinuz-3.13.0-55-generic` (kernel image)
- `/proc` (kernel data structures)

二进制文件

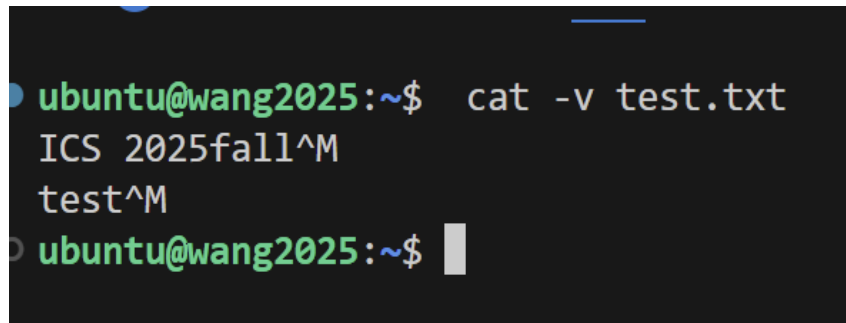
Binary File

- 研讨题1.2：二进制文件是任意字节序列，无固定文本格式；
- 文本编辑器会将二进制文件中的非文本字节（如 0x80、0xFF）强制解码为字符，但这些字节不对应任何可打印文本，因此显示为乱码。

txt文件

TXT File

- 研讨题1.2:
- Windows: 换行符为 `\r\n` ; Linux: 换行符为 `\n` (仅 LF, ASCII 码: 0xa) ;
- Linux 的 `cat -v` 会将非打印控制字符显示为 `^`+对应ASCII码的大写字母 (如 `\r` 的 ASCII 码是 13, 对应字母 M, 故显示为 `^M`) 。



文件的读写

File Read/Write

研讨题2.1

- 每次打开文件，都会有唯一非负整数 **文件描述符** 与之对应
- Shell 创建时的默认三个文件描述符：
 - 0 标准输入 `STDIN_FILENO` 键盘（行缓冲）
 - 1 标准输出 `STDOUT_FILENO` 显示屏幕（行缓冲）
 - 2 标准错误 `STDERR_FILENO` 显示屏幕（无缓冲）
- 文件的读：复制文件的内容到内存，可能会遇到 EOF（End of File）
- 文件的写：复制内存的内容到文件

文件类型

File Type

- 普通文件 `-`：文本文件和二进制文件
- 目录文件 `d`：文件夹 / 目录文件，一个目录至少含有两个条目，一个指向自己本身（`.`），一个指向其父亲（`..`）
- 套接字文件 `s`：和另外一个进程进行跨网络通信的 **文件**

```
cd /var/run
eza -l --grid # eza 是一个 ls 的替代品，功能更强大
cd ~
mkdir test && cd test
eza -laa # -l 列出文件的详细信息，-a 列出所有文件，包括隐藏文件，两个 -a 启用对 . / .. 的显示
```

不要认为套接字这个拗口的词汇和文件有什么本质的不同，它就是一层抽象，使得网络通信和文件一样会略有差异，但是概念是相通的。在学未来的网络编程时，尤其需要明确这一点。

文件路径

File Path

- 绝对路径：从根目录 `/` 开始的路径
- 相对路径：从当前目录 `.` 开始的路径

尝试运行：

```
ls / # 列出根目录下的所有文件
ls . # 列出当前目录下的所有文件
```

其他路径：

- `..` 父目录
- `~` 当前用户的主目录，在 macOS / Linux 中是 `/Users/username`

目录层次结构

Directory Hierarchy

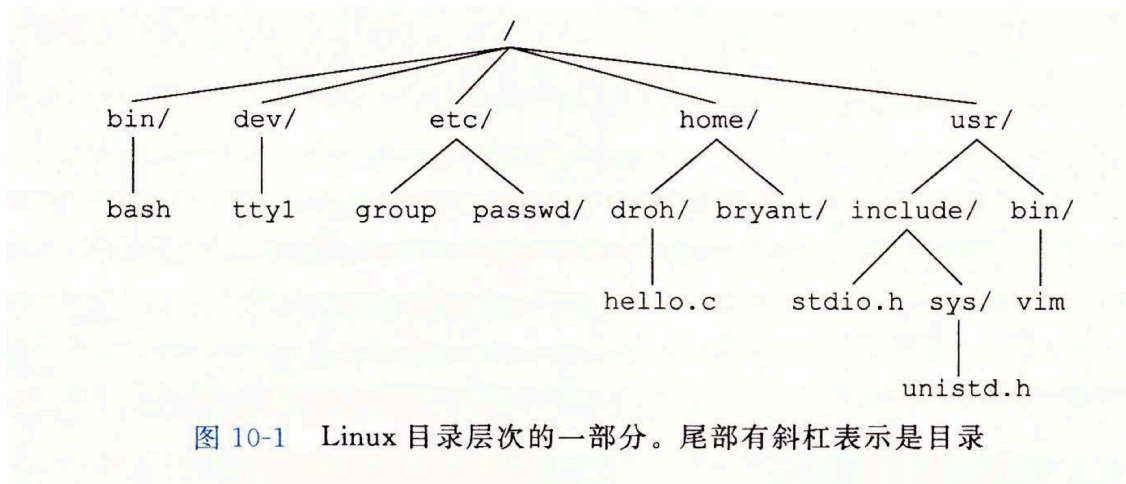


图 10-1 Linux 目录层次的一部分。尾部有斜杠表示是目录

你可以使用 `tree` 命令来查看当前目录的层次结构：

```
tree
# 如果太多，可以使用 -L 2 来限制层级
tree -L 2
```

打开文件

Open File

```
int open(const char *pathname, int flags, mode_t mode);
```

参数:

- `pathname` 文件路径, 绝对路径或当前路径下文件名 (可以使用相对路径)
- `flags` 打开文件的方式
- `mode` 新文件的权限

返回值: `int`

- 成功: 返回文件描述符
- 失败: 返回 `-1`, 并设置 `errno`

打开文件 - Flags

Open File - Flags

常量	含义
<code>O_RDONLY</code>	只读打开, Read Only
<code>O_WRONLY</code>	只写打开, Write Only
<code>O_RDWR</code>	读写打开, Read Write
<code>O_CREAT</code>	如果文件不存在, 则创建它, 需要 <code>mode</code> , Create
<code>O_TRUNC</code>	如果文件存在, 且以写模式 (<code>O_WRONLY</code> 或 <code>O_RDWR</code>) 打开, 则将其长度截断为 0, Truncate
<code>O_APPEND</code>	在每次写操作前, 设置文件位置到文件结尾处 (是写操作前而不是就在打开文件时) , Append

前三个必选其一, 后三个可选。类似上节课讲的, 我们可以用 `|` 来组合多个标志。

如 `O_RDWR | O_CREAT | O_TRUNC` 表示以读写模式打开文件, 如果文件不存在则创建它, 若存在则截断文件长度为 0。

打开文件 - Mode

Open File - Mode

新文件的访问权限位，每个进程还额外有 `umask` 掩码，用于限制新文件的权限。

当进程创建新文件时，新文件的权限位是 `mode & ~umask`。

```
eza -l ~
```

假设得到结果 `drwxrwxr-x`，其代表：

```
d | rwx | rwx | r-x
```

- 文件类型：目录
- 文件所有者权限：读、写、执行
- 文件所有者所在组权限：读、写、执行
- 其他用户权限：读、执行

打开文件 - Mode

Open File - Mode

`umask` 值是一个三位八进制数，每一位分别对应文件权限中的用户（owner）、组（group）和其他人（others）。它表示要从默认权限中屏蔽的位。

```
#include <sys/types.h>
#include <sys/stat.h>

mode_t umask(mode_t mask);
```

打开文件 - Mode

Open File - Mode

`mode` 值是一个四位八进制数，一般使用常量来指定，在创建新文件时生效，其组成包括：

- 特殊权限位
- 文件所有者权限
- 文件所有者所在组权限
- 其他用户权限

常量：

- `S_IRUSR`、`S_IWUSR`、`S_IXUSR`：用户（`USR`，`user`）对文件的读/写/执行权限。
- `S_IRGRP`、`S_IWGRP`、`S_IXGRP`：用户组（`GRP`，`group`）对文件的读/写/执行权限。
- `S_IROTH`、`S_IWOTH`、`S_IXOTH`：其他人（`OTH`，`other`）对文件的读/写/执行权限。

研讨题1.3

Q1.3

```
char* buf[30];
/* file descriptor */

int main()
{
    fd = open("/donotexist", O_RDONLY);
    printf("%d\n", fd);
    int t = read(fd, buf, 3);
    printf("%d\n", t);
    return 0;
}
```

```
ubuntu@wang2025:~/ICS25fall/test$ gcc main.c
ubuntu@wang2025:~/ICS25fall/test$ ./main
-1
-1
```

```
int fd;
/* file descriptor */

int main()
{
    if ((fd = open("/donotexist", O_RDONLY)) < 0) {
        perror("open");
        exit(1);
    }
    printf("%d\n", fd);
    return 0;
}
```

```
ubuntu@wang2025:~/ICS25fall/test$ gcc main.c
ubuntu@wang2025:~/ICS25fall/test$ ./main
open: No such file or directory
```

关闭文件

Close File

```
int close(int fd);
```

返回值: `int`

- 成功: 返回 `0`
- 失败: 返回 `-1`, 并设置 `errno`

关闭一个已经关闭的描述符会出错。

无论进程因为什么而终止, 内核都会关闭所有打开的文件并释放内存资源。

研讨题1.3

Q1.3

```
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<unistd.h>
#include<stdlib.h>
#include<stdio.h>

int main()
{
    int fd = 8;
    /* file descriptor */
    int retval; /* return value */
    retval = close(fd);
    printf("%d\n", retval);
    return 0;
}
```

```
int main()
{
    int fd = 8;
    /* file descriptor */
    int retval; /* return value */
    if ((retval = close(fd)) < 0) {
        perror("close");
        exit(1);
        printf("%d\n", retval);
    }
    return 0;
}
```

```
ubuntu@wang2025:~/ICS25fall/test$ gcc main.c
ubuntu@wang2025:~/ICS25fall/test$ ./main
close: Bad file descriptor
```

```
• ubuntu@wang2025:~/ICS25fall/test$ gcc main.c
• ubuntu@wang2025:~/ICS25fall/test$ ./main
-1
```

读和写

Read and Write

读 read

```
ssize_t read(int fd, void *buf, size_t n);
```

- `fd` 文件描述符
- `buf` 缓冲区，用于存储读取到的数据
- `n` 读取的字节数

返回值: `ssize_t` (有符号整数, signed size_t)

- 成功: **返回读取的字节数**
- 读到 EOF: 返回 `0` 也不算失败, 但也不算完全正常?
- 失败: 返回 `-1`, 并设置 `errno`

当返回非负, 但是小于预期的 n 时, 表示写入操作只成功了一部分, 我们称之为 **不足值**

写 write

```
ssize_t write(int fd, const void *buf, size_t n);
```

- `fd` 文件描述符
- `buf` 缓冲区, 用于存储写入的数据
- `n` 写入的字节数

返回值: `ssize_t` (有符号整数, signed size_t)

- 成功: **返回写入的字节数**
- 失败: 返回 `-1`, 并设置 `errno`

不足值

Short Count

不足值： $-1 < \text{ret} < n$

不足值的出现原因：

- 读到 EOF：（假设 $n = 50$ ）此次返回到 EOF 时已读取的数量 < 50 ，比如 20，下次再读取时返回 0
- 从终端传输文本行：一般设置读取的字节数较大，但是每次读取的文本行可能较短，所以返回的字节数会小于请求的字节数
- 读和写网络套接字（Socket）：如果打开的文件对应于网络套接字，那么内部缓冲约束和较长的网络延迟会引起 `read` 和 `write` 返回不足值（回忆一下信号中断）

由于不足值的存在，为了保证输入输出能够正常的、按照预期的完成（主要是情况 3），我们可能需要反复、多次调用 `read` 和 `write`。

研讨题1.4

Q1.4

```
//main.c
#include "csapp.h"
int main(int argc, char *argv[])
{
    char c;
    int infd = STDIN_FILENO;
    if (argc == 2) {
        infd = Open(argv[1], O_RDONLY, 0);
    }
    while(Read(infd, &c, 1) != 0)
        Write(STDOUT_FILENO, &c, 1);
    exit(0);
}
```

```
//test.txt
ICS2025fall
```

```
● ubuntu@wang2025:~/ICS25fall/test$ ./main
ICS2025fall
ICS2025fall
a
a
● ubuntu@wang2025:~/ICS25fall/test$ ./main test.txt
ICS2025fall
```

- 一个 echo 程序
- 可以选择文件/键盘输入

研讨题1.4&2.2

Q1.4&2.2

```
//main.c
#include "csapp.h"
#define BUFSIZE 64
int main(int argc, char *argv[])
{
    int nread;
    char buf[BUFSIZE];
    int infd = STDIN_FILENO;
    if (argc == 2) {
        infd = Open(argv[1], O_RDONLY, 0);
    }
    while((nread = Read(infd, buf, BUFSIZE)) != 0)
        Write(STDOUT_FILENO, buf, nread);
    exit(0);
}

//test.txt
ICS2025fall
```

```
ubuntu@wang2025:~/ICS25fall/test$ strace ./main test.txt
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RL
munmap(0x751afe8bf000, 24823) = 0
openat(AT_FDCWD, "test.txt", O_RDONLY) = 3
read(3, "ICS2025fall\n", 64) = 12
write(1, "ICS2025fall\n", 12ICS2025fall
) = 12
read(3, "", 64) = 0
exit_group(0) = ?
+++ exited with 0 +++
```

- 对比上一页的程序

- 使用缓冲区减少系统调用次数，标准I/O函数使用缓冲区

标准 I/O

Standard I/O

研讨题 2.1

功能分类	代表函数	作用描述
打开/关闭	<code>fopen()</code> 、 <code>fclose()</code> 、 <code>fdopen()</code>	<code>fopen</code> ：打开文件返回 FILE 流； <code>fdopen</code> ：将 fd 转换为 FILE； <code>fclose</code> ：关闭流并刷新缓冲
字节读写	<code>fread()</code> 、 <code>fwrite()</code>	从流读写指定字节数（适合二进制文件）
行读写	<code>fgets()</code> 、 <code>fputs()</code>	<code>fgets</code> ：读取一行（直到 '\n' 或 EOF） <code>fputs</code> ：写入一行
格式化读写	<code>fscanf()</code> 、 <code>fprintf()</code> 、 <code>sscanf()</code> 、 <code>sprintf()</code>	格式化输入输出 <code>sscanf/sprintf</code> 针对字符串操作

标准 I/O 缓冲

Standard I/O

研讨题 2.3

```
#include <stdio.h>

int main()
{
    printf("h");
    printf("e");
    printf("l");
    printf("l");
    printf("o");
    printf("\n");
    fflush(stdout);
    exit(0);
}
```

```
linux> strace ./hello
execve("./hello", ["hello"], [/* ... */]).
...
write(1, "hello\n", 6)           = 6
...
exit_group(0)                   = ?
```

- `\n` 自动触发 `fflush`
- 减少系统调用次数（需要内核态用户态切换）

补充

Standard I/O

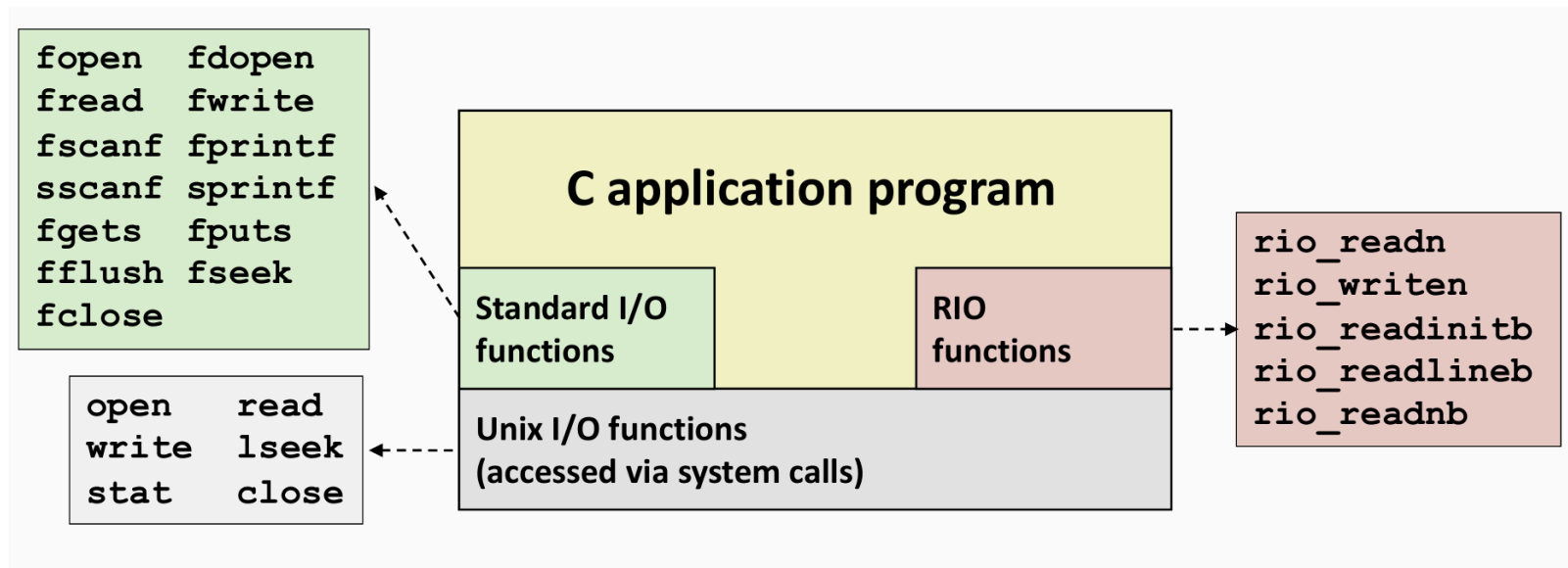
- 缓冲区内容会被子进程继承

```
#include "csapp.h"
int main()
{
    printf("with fflush ");
    fflush(stdout);
    printf("without fflush ");
    fork();
    return 0;
}
```

```
● ubuntu@wang2025:~/ICS25fall/test$ gcc csapp.c main.c
● ubuntu@wang2025:~/ICS25fall/test$ ./main
○ with fflush without fflush without fflush ubuntu@wang
```

研讨题2.4

Q2.4



- 编写好的使用系统调用的 C 函数
- 直接使用这些函数，不用处理系统调用被打断/返回值等

RIO 包

Robust I/O

让你的 I/O 健壮一点~

- good coding practice: handles error checking, signals, and “short counts”
- 研讨题 2.5

RIO - 不带缓冲的读

`rio_readn`

`rio_readn` 函数用于从文件描述符 `fd` 中读取 `n` 个字节到缓冲区 `usrbuf` 中。

它是 **无缓冲** 的，与 `read` 相比，多了对不足值的处理。

注意鉴别这里的 **无缓冲** 和参数之一的 **缓冲区**。

后者的定义是，内存中即将存储读入数据的区域。

前者的定义是，有没有对 `read` 这个函数本身实现缓冲机制（我会在后面将带缓冲的版本让大家对比理解）。

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);        /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的读

rio_readn

首先，设置变量：

- `nleft` 为请求的字节数
- `bufp` 为缓冲区指针
- `nread` 变量用于存储每次读取的字节数

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* 返回值 >= 0 */
}
```


RIO - 不带缓冲的读

rio_readn

循环读入（多次尝试），正常情况下唯一的退出条件是 `nleft <= 0`。

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的读

rio_readn

每次调用 `read` 来尝试读入 `nleft` 个字节。

`nread` 中会存储实际读入的字节数。

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的读

`rio_readn`

当 `nread` 小于 0 时，说明返回值是 `-1`，代表出错。

如果 `errno` 被设置为 `EINTR`，表示被信号处理程序中断（`interrupt`）返回，此时需要将 `nread` 设置为 0。

这是因为，`errno == EINTR` 表示这次调用确实没有读取任何数据（如缺页异常），文件指针没有发生变化，我们即将重试此次操作。

如果 `errno` 不是 `EINTR`，则说明读取失败是由于 `read()` 本身导致的，此时我们放弃继续读取，返回代表出错的 `-1`，并保留 `errno` 不变。一些可能情况：

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的读

rio_readn

回忆我们刚才说的，当遇到 EOF 时，read 会先返回此次读取到的字节数，再次调用 read 则会返回 0。

这两句判断就对应再次调用 read 的情况。

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);        /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的读

rio_readn

望文生义，我们还需要读取的字数是 `nleft - nread`，文件指针也要移动 `nread` 个字节。

还是举 EOF 那个情况：当遇到 EOF 时，`read` 会先返回此次读取到的字节数，再次调用 `read` 则会返回 0。

这对应先前那次调用 `read` 的情况。

（但其实这个例子不是唯一的，涉及套接字的话，可能多次返回，每次返回的 `nread` 都只是 `nleft` 的一部分）

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的读

rio_readn

运行到这的唯一两种可能：

- `nleft == 0`，完全读入成功，返回 `n`
- EOF 导致退出，返回 `n - nleft`，即实际读入的字节数。

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nread = 0;      /* 再次调用 read() */
            else
                return -1;      /* read() 设置了 errno */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }

    return (n - nleft);        /* 返回值 >= 0 */
}
```

RIO - 不带缓冲的写

rio_writen

`rio_writen` 函数用于从文件描述符 `fd` 中写入 `n` 个字节到缓冲区 `usrbuf` 中。

它是 **无缓冲** 的，与 `write` 相比，多了对不足值的处理。

和 `readn` 行为类似，但是没有了对于 EOF 的处理，所以永远不会返回不足值（要么返回错误 `-1`，要么返回 `n`）。

```
ssize_t rio_writen(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nwritten;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nwritten = write(fd, bufp, nleft)) <= 0) {
            if (errno == EINTR) /* 被信号处理程序中断返回 */
                nwritten = 0; /* 再次调用 write() */
            else
                return -1; /* write() 设置了 errno */
        }
        nleft -= nwritten;
        bufp += nwritten;
    }
    return n; /* 返回原始请求的字节数 */
}
```

RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

函数组合定义:

- `n` 是读 `n` 个字节
- `b` 是 `buffer` 的意思, 代表使用缓冲区

```
1 // rio_t - 自定义的带缓冲区的读入结构体
2 typedef struct {
3     int rio_fd;           /* 描述符 */
4     ssize_t rio_cnt;      /* 缓冲区中未读字节数 */
5     char *rio_bufptr;     /* 下一个未读字节 */
6     char rio_buf[RIO_BUFSIZE]; /* 缓冲区 */
7 } rio_t;
8
9 // rio_readinitb - 初始化rio_t
10 void rio_readinitb(rio_t *rp, int fd)
11 {
12     rp->rio_fd = fd;
13     rp->rio_cnt = 0;
14     rp->rio_bufptr = rp->rio_buf;
15 }
16
17 // rio_read - 健壮地读入 n 个字节, 带缓冲区
18 static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n) {
19     int cnt;
20     while (rp->rio_cnt <= 0) { /* 重新填充缓冲区 */
21         rp->rio_cnt = read(rp->rio_fd, rp->rio_buf, sizeof(rp->rio_b
22         if (rp->rio_cnt < 0) {
```


RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

带缓冲什么意思呢？

假设我们已经知道，要以连续进行很多次 `read` 操作，每次读 n 个字节。

那么，考虑如下两种方式：

1. 调用多次 `read` 函数，每次读取 n 个字节。
2. 一次读取很多字节到内存，然后每次读的时候到内存中取 n 个字节。

这两种方式，后者由于可以直接在内存中取数据，**减少实际系统调用**

```

1 // rio_t - 自定义的带缓冲区的读入结构体
2 typedef struct {
3     int rio_fd; /* 描述符 */
4     ssize_t rio_cnt; /* 缓冲区中未读字节数 */
5     char *rio_bufptr; /* 下一个未读字节 */
6     char rio_buf[RIO_BUFSIZE]; /* 缓冲区 */
7 } rio_t;
8
9 // rio_readinitb - 初始化rio_t
10 void rio_readinitb(rio_t *rp, int fd)
11 {
12     rp->rio_fd = fd;
13     rp->rio_cnt = 0;
14     rp->rio_bufptr = rp->rio_buf;
15 }
16
17 // rio_read - 健壮地读入 n 个字节，带缓冲区
18 static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n) {
19     int cnt;
20     while (rp->rio_cnt <= 0) { /* 重新填充缓冲区 */
21         rp->rio_cnt = read(rp->rio_fd, rp->rio_buf, sizeof(rp->rio_b
22         if (rp->rio_cnt < 0) {

```

RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

`rio_t`

为了能够实现这个涉及，我们需要声明一个结构体 `rio_t`，用于存储缓冲区信息：

- `rio_fd` 文件描述符
- `rio_cnt` 缓冲区中未读字节数
- `rio_bufptr` 指向缓冲区内下一个未读字节的指针
- `rio_buf` 缓冲区本体

定义函数将结构体初始化
缓冲区的意义：**减少对于** `read` **的系统调用次数。**

```

1 // rio_t - 自定义的带缓冲区的读入结构体
2 typedef struct {
3     int rio_fd;           /* 描述符 */
4     ssize_t rio_cnt;      /* 缓冲区中未读字节数 */
5     char *rio_bufptr;     /* 下一个未读字节 */
6     char rio_buf[RIO_BUFSIZE]; /* 缓冲区 */
7 } rio_t;
8
9 // rio_readinitb - 初始化rio_t
10 void rio_readinitb(rio_t *rp, int fd)
11 {
12     rp->rio_fd = fd;
13     rp->rio_cnt = 0;
14     rp->rio_bufptr = rp->rio_buf;
15 }
16
17 // rio_read - 健壮地读入 n 个字节，带缓冲区
18 static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n) {
19     int cnt;
20     while (rp->rio_cnt <= 0) { /* 重新填充缓冲区 */
21         rp->rio_cnt = read(rp->rio_fd, rp->rio_buf, sizeof(rp->rio_b
22         if (rp->rio_cnt < 0) {

```

RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

`rio_read`

这个函数和先前讲过的

`rio_readn` 函数有一些类似，支持不足值的处理。

且在此之上，它支持了缓冲区的设计，通过移动 `rp->rio_bufptr` 而不是使用 `nleft` 来指明当前读取到的位置。

```

34     memcpy(usrbuf, rp->rio_bufptr, cnt);
35     rp->rio_bufptr += cnt;
36     rp->rio_cnt -= cnt;
37     return cnt;
38 }
39
40 // rio_readnb - 健壮地读入 n 个字节，带缓冲区
41 ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n) {
42     size_t nleft = n;
43     ssize_t nread;
44     char *bufp = usrbuf;
45     while (nleft > 0) {
46         if ((nread = rio_read(rp, bufp, nleft)) < 0) {
47             if (errno == EINTR) /* 由于信号处理程序返回而中断 */
48                 nread = 0;      /* 再次调用rio_read()函数 */
49             else
50                 return -1; /* 由rio_read()设置的errno */
51         } else if (nread == 0)
52             break; /* EOF */
53         nleft -= nread;
54         bufp += nread;
55     }

```

RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

`rio_read`

重点关注在 13 行，它真实调用 `read` 函数的时候，并不是读了 `n` 个字节，而是读了 `sizeof(rp->rio_buf)` 个字节。

这代表它会尝试将缓冲区填满，而不是仅仅读取 `n` 个字节。

26 行，`memcpy` 函数将 `rio_t` 内缓冲区的数据复制到实际想要存到的 `usrbuf` 中。

```

34     memcpy(usrbuf, rp->rio_bufptr, cnt);
35     rp->rio_bufptr += cnt;
36     rp->rio_cnt -= cnt;
37     return cnt;
38 }
39
40 // rio_readnb - 健壮地读入 n 个字节，带缓冲区
41 ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n) {
42     size_t nleft = n;
43     ssize_t nread;
44     char *bufp = usrbuf;
45     while (nleft > 0) {
46         if ((nread = rio_read(rp, bufp, nleft)) < 0) {
47             if (errno == EINTR) /* 由于信号处理程序返回而中断 */
48                 nread = 0;      /* 再次调用rio_read()函数 */
49             else
50                 return -1; /* 由rio_read()设置的errno */
51         } else if (nread == 0)
52             break; /* EOF */
53         nleft -= nread;
54         bufp += nread;
55     }

```

RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

`rio_readnb`

这个函数和先前讲过的
`rio_readn` 函数完全类似，只不过迁移到了缓冲区的实现。

注意第 38 行，这里用的是刚才讲的 `rio_read` 函数，而不是 `read` 函数。

这代表，它读取的目标是缓冲区，而不是文件。

```
60  ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen) {
61      int n, rc;
62      char c, *bufp = usrbuf;
63
64      for (n = 1; n < maxlen; n++) {
65          if ((rc = rio_read(rp, &c, 1)) == 1) {
66              *bufp++ = c;
67              if (c == '\n')
68                  break;
69          } else if (rc == 0) {
70              if (n == 1)
71                  return 0; /* EOF, 没有读入任何数据 */
72              else
73                  break; /* EOF, 读入了部分数据 */
74          } else
75              return -1; /* 错误 */
76      }
77      *bufp = 0;
78      return n;
79  }
```

RIO - 带缓冲的读

`rio_read` / `rio_readnb` / `rio_readlineb`

`rio_readlineb` 函数从文件 `rp` 读出下一个文本行（包括结尾的换行符），将它复制到内存位置 `usrbuf`，并且用 `NULL`（零）字符来结束这个文本行。

`rio_readlineb` 函数最多读 `maxlen-1` 个字节，余下的一个字符留给结尾的 `NULL` 字符。

超过 `maxlen-1` 字节的文本行会被截断。

由于核心的 57 行实现是基于 `rio_read` 函数的，所以我们可以肆无忌惮地使用它，每次只读 1

```
60 ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen) {
61     int n, rc;
62     char c, *bufp = usrbuf;
63
64     for (n = 1; n < maxlen; n++) {
65         if ((rc = rio_read(rp, &c, 1)) == 1) {
66             *bufp++ = c;
67             if (c == '\n')
68                 break;
69         } else if (rc == 0) {
70             if (n == 1)
71                 return 0; /* EOF, 没有读入任何数据 */
72             else
73                 break; /* EOF, 读入了部分数据 */
74         } else
75             return -1; /* 错误 */
76     }
77     *bufp = 0;
78     return n;
79 }
```

RIO 总结

Robust I/O Summary

缓冲区的实现：基于 `rio_t` 结构体，在此基础上封装出了 `rio_read` / `rio_readinitb` / `rio_readnb` / `rio_readlineb` 函数。

缓冲区的思想：通过一次读入很多字节，实现一个缓存机制，后续再读直接从缓存中取，从而减少系统调用次数。

不带缓冲区的实现：实现了对不足值的处理，包括 EOF / 系统调用中断等，在此基础上封装出了 `rio_readn` / `rio_writen` 函数。

带缓冲的版本和不带缓冲的版本，在实现上有很多类似的地方，**但一定不能混用。**

例：`rio_readlineb` 和 `rio_readnb` 可以混用，但不能和 `rio_readn` 混用。

文件元数据

File Metadata

```
int stat(const char *filename, struct stat *buf);  
int fstat(int fd, struct stat *buf);
```

stat 结构体

```
/* Metadata returned by the stat and fstat functions */  
struct stat {  
    dev_t      st_dev;      /* Device */  
    ino_t      st_ino;      /* inode */  
    mode_t     st_mode;     /* Protection and file type */  
    nlink_t    st_nlink;    /* Number of hard links */  
    uid_t      st_uid;      /* User ID of owner */  
    gid_t      st_gid;      /* Group ID of owner */  
    dev_t      st_rdev;     /* Device type (if inode device) */  
    off_t      st_size;     /* Total size, in bytes */  
    unsigned long st_blksize; /* Block size for filesystem I/O */  
    unsigned long st_blocks;  /* Number of blocks allocated */  
    time_t     st_atime;     /* Time of last access */  
    time_t     st_mtime;     /* Time of last modification */  
    time_t     st_ctime;     /* Time of last change */  
};
```


文件元数据

File Metadata

- `st_size` 成员包含了文件的字节数大小
- `st_mode` 成员则编码了文件访问许可位和文件类型
 - `S_ISREG(m)` 这是一个普通文件吗？
 - `S_ISDIR(m)` 这是一个目录文件吗？
 - `S_ISSOCK(m)` 这是一个网络套接字吗？

文件元数据

File Metadata

研讨题1.5 —— 宏谓词的用法

```
int main (int argc, char **argv)
{
    struct stat stat;
    char *type, *readok;
```

```
    Stat(argv[1], &stat);
    if (S_ISREG(stat.st_mode))      /* Determine file type */
        type = "regular";
    else if (S_ISDIR(stat.st_mode))
        type = "directory";
    else
        type = "other";
    if ((stat.st_mode & S_IRUSR)) /* Check read access */
        readok = "yes";
    else
        readok = "no";
```

```
    printf("type: %s, read: %s\n", type, readok);
    exit(0);
}
```

statcheck.c

```
linux> ./statcheck statcheck.c
type: regular, read: yes
linux> chmod 000 statcheck.c
linux> ./statcheck statcheck.c
type: regular, read: no
linux> ./statcheck ..
type: directory, read: yes
```

Chmod 指令

Chmod

- chmod 作用：修改文件权限 (change mode)
- 000 含义：八进制权限表示法，三个数字分别对应「所有者 (u)、组用户 (g)、其他用户 (o)」，每个数字的二进制位对应 r (读, 4)、w (写, 2)、x (执行, 1)



- 报错 `bash: ./runtrace: Permission denied`
- `chmod +x` 相当于 `chmod a+x`
- 给所有用户添加执行权限

Chmod 指令

Chmod

研讨题1.5

- 数字范围：000 ~ 777
- 常见合法值：

八进制值	权限符号	含义（所有者 + 组 + 其他）	适用场景
100	r-- — —	所有者仅可读	普通配置文件
644	rw- r-- r--	所有者读写，其他只读	普通文本文件、源代码
755	rwx r-x r-x	所有者读写执行，其他读执行	可执行程序、脚本、目录
777	rwx rwx rwx	所有用户全权限	临时共享文件

读取目录内容

Reading Directory Contents

```
DIR *opendir(const char *name);
// 返回: 若成功, 则为处理的指针; 若出错, 则为 NULL。
struct dirent *readdir(DIR *dirp);
// 返回: 若成功, 则为指向下一个目录项的指针;
// 若没有更多的目录项或出错, 则为 NULL。
int closedir(DIR *dirp); // 返回: 成功为 0; 错误为 -1。
```

- opendir 返回指向目录流的指针
- 区分 readdir 流结束还是出错: 看 errno 是否被修改过
- dirent 结构

```
struct dirent{
    ino_t d_ino; //文件位置
    char d_name[256]; //文件名
}
```

```
#include "csapp.h"

int main(int argc, char **argv)
{
    DIR *stream;
    struct dirent *dep;

    if (argc != 2) {
        printf("usage: %s <pathname>\n", argv[0]);
        exit(1);
    }
    stream = opendir(argv[1]);

    errno = 0;
    while ((dep = readdir(stream)) != NULL) {
        printf("Found file: %s\n", dep->d_name);
    }
    if (errno != 0)
        unix_error("readdir error");
    Closedir(stream);
    exit(0);
}
```

读取目录内容

Reading Directory Contents

运行示例:

```
> ICS24fall
✓ ICS25fall
  > lab1
  > lab3
  > lab4
  > lab4test
  ✓ test
    C csapp.c
    C csapp.h
    ≡ main
    C main.c
    ≡ nobuf
    ≡ test.txt
    ≡ test2.txt
  > network-lab
```

```
● ubuntu@wang2025:~/ICS25fall/test$ ./main ./
Found file: csapp.h
Found file: nobuf
Found file: csapp.c
Found file: main
Found file: main.c
Found file: test2.txt
Found file: test.txt
Found file: ..
Found file: .

● ubuntu@wang2025:~/ICS25fall/test$ ./main ../
Found file: lab4
Found file: lab1
Found file: lab4test
Found file: ..
Found file: lab3
Found file: test
Found file: .
```

共享文件

Shared Files

研讨题1.6

数据结构	描述	共享性	关键内容
描述符表	每个进程都有独立的描述符表	不共享	描述符表项由文件描述符索引，指向文件表项
文件表	表示打开文件的集合	所有进程共享	文件位置、引用计数、v-node 指针
v-node 表	包含文件的 <code>stat</code> 结构信息	所有进程共享	<code>st_mode</code> 和 <code>st_size</code>

这么类比着记：

- 描述表，每个进程单独一个，符合直觉
- v-node 是类的声明，一个真实文件只有一个，所有进程共享
- 打开文件表是类的实例，所有进程共享，每个实例必然有与之对应的声明（v-node），但是多个实例可以指向同一个声明（v-node）
- 多个进程可以共用同一个文件表，类比他们共用相同的实例。

共享文件

Shared Files

打开文件表和 v-node 表的最核心差异：

- 打开文件表的每一项 **对应一次打开**
- v-node 表的每一项 **对应一个文件**

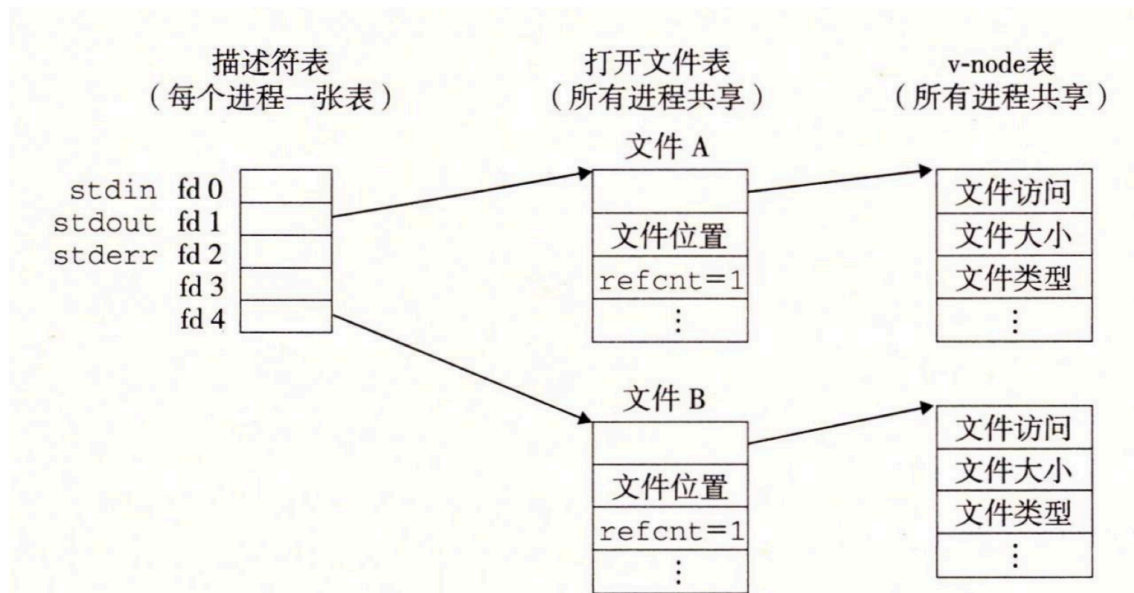


图 10-12 典型的打开文件的内核数据结构。在这个示例中，两个描述符引用不同的文件。没有共享

共享文件

Shared Files

打开文件表和 v-node 表的最核心差异：

- 打开文件表的每一项 **对应一次打开**
- v-node 表的每一项 **对应一个文件**

由于一个文件可以打开多次，所以可以有多个打开文件表项指向同一个 v-node 表项。

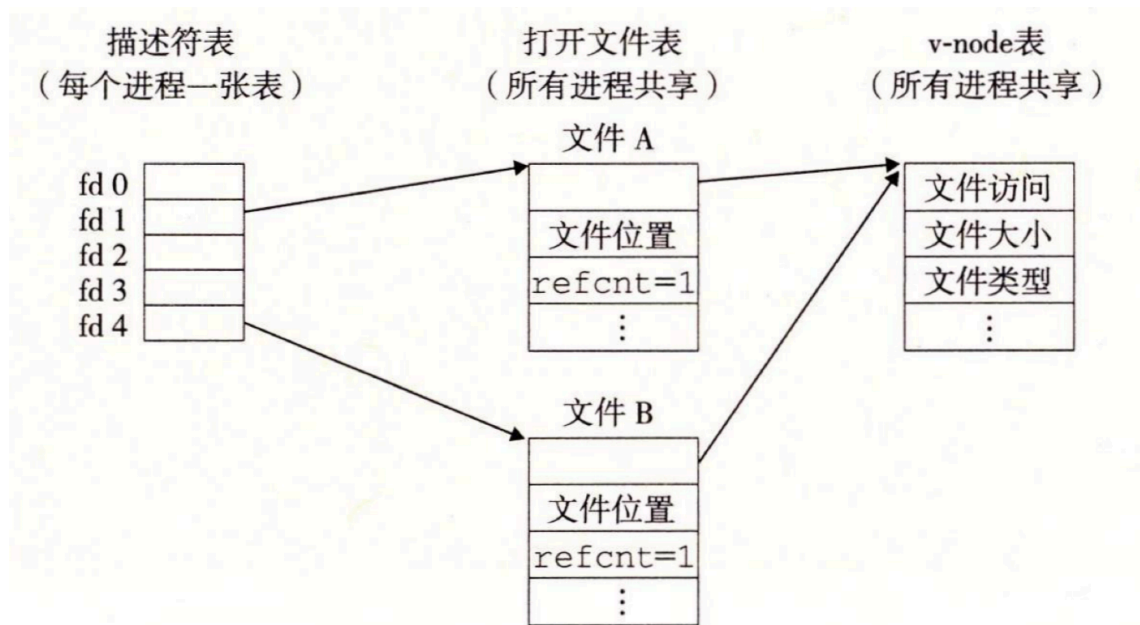


图 10-13 文件共享。这个例子展示了两个描述符通过两个打开文件表表项共享同一个磁盘文件

共享文件

Shared Files

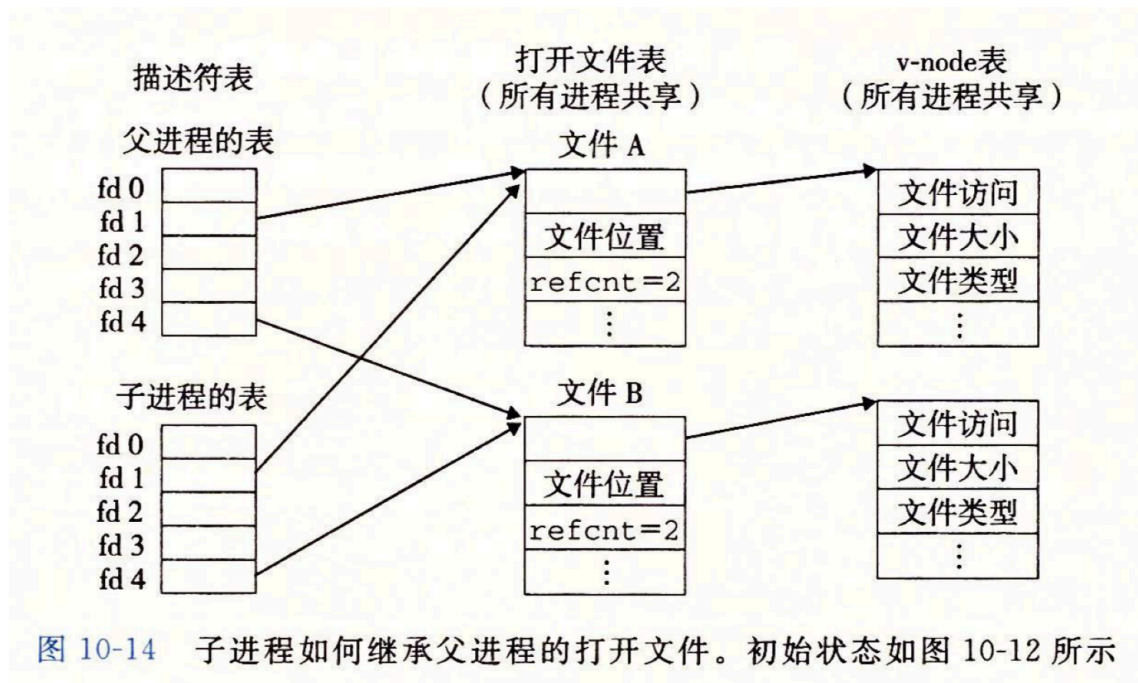
打开文件表和 v-node 表的最核心差异：

- 打开文件表的每一项 **对应一次打开**
- v-node 表的每一项 **对应一个文件**

由于一个文件可以打开多次，所以可以有多个打开文件表项指向同一个 v-node 表项。

研讨题1.7

由于多个文件可以共享一次打开，所以有了打开文件表中的 `refcnt`，多个进程的描述表中的描述符可以对应同一个打开文件表项（如父子进程）。



共享文件 - 总结

Shared Files - Summary

描述符表

- 每个进程都有独立的描述符表
- 描述符表的表项由进程打开的文件描述符来索引
- 每个打开的描述符表项指向文件表中的一个表项
- **对应一个进程所需要的描述符信息**

文件表

- 打开的文件集合由文件表表示，所有进程共享该表。
- 文件表的表项包括：
 - 文件位置：当前文件位置
 - 引用计数 `refcnt`：当前指向该表项的描述符表项数量
 - v-node 指针：指向 v-node 表中对应表项的指针
- 关闭一个描述符会减少相应文件表表项的引用计数
- 内核不会删除文件表表项，直到引用计数为零
- **对应一次打开**

v-node 表

- v-node 表同样由所有进程共享
- 每个表项包含 `stat` 结构中的大多数信息，包括 `st_mode` 和 `st_size` 成员
- **对应一个文件**

I/O 重定向

I/O Redirection

dup

```
int dup(int oldfd);
```

dup 用于复制一个文件描述符 `oldfd`，并返回一个新的文件描述符。

新的文件描述符与原来的文件描述符共享同一个文件表项。

返回值

- 成功时，返回新的文件描述符。
- 失败时，返回 `-1`，并设置 `errno` 以指示错误。

dup2

```
int dup2(int oldfd, int newfd);
```

dup2 复制文件描述符 `oldfd` 到 `newfd`。

注意顺序：是用前面的覆盖后面的，也就是把后面的指向前面

如果 `newfd` 已经打开，它会首先被关闭。

返回值

- 成功时，返回 `newfd`。
- 失败时，返回 `-1`，并设置 `errno` 以指示错误。

I / O 重定向

I / O Redirection

研讨题1.8

研讨题1.9

Q1.9

```
//main.c
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1, fd2, fd3;
    char c1, c2, c3;
    char *fname = argv[1];
    fd1 = Open(fname, O_RDONLY, 0);
    fd2 = Open(fname, O_RDONLY, 0);
    fd3 = Open(fname, O_RDONLY, 0);
    Dup2(fd2, fd3);
    Read(fd1, &c1, 1);
    Read(fd2, &c2, 1);
    Read(fd3, &c3, 1);
    printf("c1 = %c, c2 = %c, c3 = %c\n", c1, c2, c3);
    return 0;
}

//test.txt
abcde
```

```
ubuntu@wang2025:~/ICS25fall/test$ ./main test.txt
c1 = a, c2 = a, c3 = b
ubuntu@wang2025:~/ICS25fall/test$
```

- fd2和fd3共享打开文件表，因此文件位置相同，fd1单独
- 三个共享同一个v-node表

研讨题1.9

Q1.9

```
//main.c
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1;
    int s = getpid() & 0x1;
    char c1, c2;
    char *fname = argv[1];
    fd1 = Open(fname, O_RDONLY, 0);
    Read(fd1, &c1, 1);
    if (fork()) { /* Parent */
        sleep(s);
        Read(fd1, &c2, 1);
        printf("Parent: c1 = %c, c2 = %c\n", c1, c2);
    } else { /* Child */
        sleep(1-s);
        Read(fd1, &c2, 1);
        printf("Child: c1 = %c, c2 = %c\n", c1, c2);
    }
    return 0;
}
```

```
//test.txt
abcde
```

```
ubuntu@wang2025:~/ICS25fall/test$ ./main test.txt
Child: c1 = a, c2 = b
Parent: c1 = a, c2 = c
ubuntu@wang2025:~/ICS25fall/test$ ./main test.txt
Parent: c1 = a, c2 = b
ubuntu@wang2025:~/ICS25fall/test$ Child: c1 = a, c2 = c
```

- 与主进程pid的奇偶相关
- 父子进程共享同一个打开文件表

研讨题2.6

Q2.6

1. 只要有可能就使用标准 I/O
 - 磁盘、终端设备 I/O
2. 不要使用 `scanf` 或 `rio_readlineb` 来读二进制文件
 - 否则会把 `0xa` 当成换行符
3. 在信号处理函数中使用 Unix I/O
4. 对网络套接字的 I/O 使用 RIO 函数
 - 需要格式化输入时先用 `rio_readlineb` 读文本行，再调用 `sscanf` 提取不同字段；格式化输出则先用 `sprintf` 格式化字符串，再调用 `rio_writen` 发送到套接口。

研讨题2.6

Why?

1. 跟在输出函数之后的输入函数

- 如果中间没有插入对 `fflush`、`fseek`、`fsetpos` 或者 `rewind` 的调用，一个输入函数不能跟随在一个输出函数之后。`fflush` 函数清空与流相关的缓冲区。后三个函数使用 Unix I/O `lseek` 函数来重置当前的文件位置。
- 在每个输入操作前刷新缓冲区

2. 跟在输入函数之后的输出函数

- 如果中间没有插入对 `fseek`、`fsetpos` 或者 `rewind` 的调用，一个输出函数不能跟随在一个输入函数之后，除非该输入函数遇到了一个文件结束。
- 对同一个打开的套接字描述符打开两个流，一个用来读，一个用来写
- 但依然会出现，需要关闭两次所产生的失败操作(线程化程序有问题)

研讨题2.7

Q2.7

```
//main.c
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1, fd2, fd3;
    char *fname = argv[1];
    fd1 = Open(fname, O_CREAT|O_TRUNC|O_RDWR,
        S_IRUSR|S_IWUSR);
    Write(fd1, "pqrs", 4);
    fd3 = Open(fname, O_APPEND|O_WRONLY, 0);
    Write(fd3, "jklmn", 5);
    fd2 = dup(fd1); /* Allocates descriptor */
    Write(fd2, "wxyz", 4);
    Write(fd3, "ef", 2);
    return 0;
}
```

```
ubuntu@wang2025:~/ICS25fall/test$ ./main test2.txt
ubuntu@wang2025:~/ICS25fall/test$ cat test2.txt
pqrsxyznefubuntu@wang2025:~/ICS25fall/test$
```

- 注意有没有 APPEND
- 关注每个文件描述符对应的打开文件表里面的位置

Emphasis

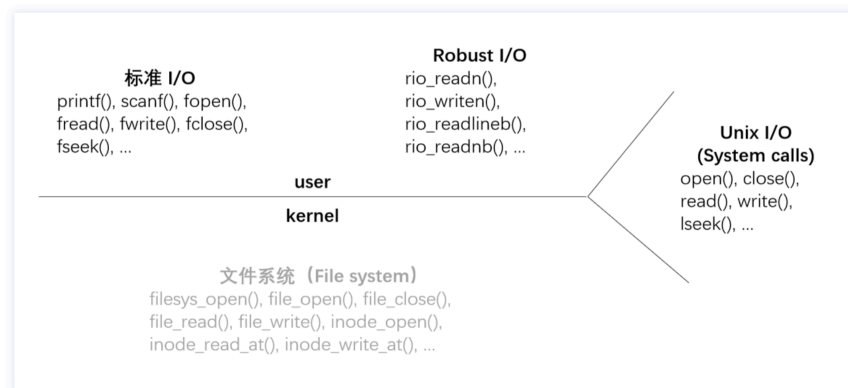
Notes

■ Note Link

- 系统级I/O
 - I/O基本概念
- 文件操作
 - 打开文件
 - 关闭文件
 - 改变位置
 - 读写文件
 - 读取文件元数据
 - 读取目录内容
- 文件类型
- 共享文件
 - 打开文件
 - 文件共享
 - 父子文件共享
- I/O重定向
- I/O包
 - Unix I/O
- RIO
 - 无缓冲I/O
 - 带缓冲I/O
- 标准I/O
- 综合使用
 - 基本原则
- 缓冲
 - 缓冲区注意
 - 缓冲与缓存
- 缓冲的问题
 - 标准I/O的限制

系统级I/O

I/O基本概念



I/O 设备又叫做输入/输出设备，它是人类用来和计算机进行通信的外部硬件。输入/输出设备能够向计算机发送数据（输出）并从计算机接收数据（输入）。

Homework Review

HW1

* 10.6 下面程序的输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6
7      fd1 = Open("foo.txt", O_RDONLY, 0);
8      fd2 = Open("bar.txt", O_RDONLY, 0);
9      Close(fd2);
10     fd2 = Open("baz.txt", O_RDONLY, 0);
11     printf("fd2 = %d\n", fd2);
12     exit(0);
13 }
```

fd2 = 4

HW2

**** 10.10** 修改图 10-5 中的 `cpfile` 程序，使得它有一个可选的命令行参数 `infile`。如果给定了 `infile`，那么复制 `infile` 到标准输出，否则像以前那样复制标准输入到标准输出。一个要求是对于两种情况，你的解答都必须使用原来的复制循环(第 9~11 行)。只允许你插入代码，而不允许更改任何已经存在的代码。

code/io/cpfile.c

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      int n;
6      rio_t rio;
7      char buf[MAXLINE];
8
9      Rio_readinitb(&rio, STDIN_FILENO);
10     while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
11         Rio_writen(STDOUT_FILENO, buf, n);
12 }
```

code/io/cpfile.c

图 10-5 从标准输入复制一个文本文件到标准输出

HW2

- 在7-9行之间插入

```
if(argc == 2)
{
    int fd;
    fd = Open(argv[1], O_RDONLY, 0);
    Dup2(fd , STDIN_FILENO);
    Close(fd);
}
```

- 可选在结束后将 `STDIN_FILENO` 恢复

Exercises

E1

13. 下列关于系统 I/O 的说法中，正确的是：

- A. Linux shell 创建的每个进程开始时都有三个打开的文件：标准输入（文件描述符为 0）、标准输出（文件描述符为 1）、标准错误（文件描述符为 2），这使得程序始终不能使用保留的描述符 0、1、2 读写其他文件。
- B. Unix I/O 的 read/write 函数是异步信号安全的，故可以在信号处理函数中使用。
- C. RIO 函数包的健壮性保证了对于同一个文件描述符，任意顺序调用 RIO 包中的任意函数不会造成问题。
- D. 使用 `int fd1 = open("ICS.txt", O_RDWR);` 打开 ICS.txt 文件后，再用 `int fd2 = open("ICS.txt", O_RDWR);` 再次打开文件，会使得 fd1 对应的打开文件表中的引用计数 `refcnt` 加一。

E1

13. 下列关于系统 I/O 的说法中，正确的是：

- A. Linux shell 创建的每个进程开始时都有三个打开的文件：标准输入（文件描述符为 0）、标准输出（文件描述符为 1）、标准错误（文件描述符为 2），这使得程序始终不能使用保留的描述符 0、1、2 读写其他文件。
- B. Unix I/O 的 read/write 函数是异步信号安全的，故可以在信号处理函数中使用。
- C. RIO 函数包的健壮性保证了对于同一个文件描述符，任意顺序调用 RIO 包中的任意函数不会造成问题。
- D. 使用 `int fd1 = open("ICS.txt", O_RDWR);` 打开 ICS.txt 文件后，再用 `int fd2 = open("ICS.txt", O_RDWR);` 再次打开文件，会使得 fd1 对应的打开文件表中的引用计数 `refcnt` 加一。

答案：B（简单）

- A. 描述符 0，1，2 可以重定向到其他文件。
- B. 正确，教材 P534。
- C. 有缓冲区和无缓冲区的不可以交叉使用，教材 P629。
- D. 多次打开文件应该对应着多个打开文件表，教材 P635。

E2

14. 考虑以下代码，假设 ICS.txt 中的初始内容为"ICS!!!ics!!!"。

```
int fd = open("ICS.txt", O_RDWR | O_CREAT | O_TRUNC,  
S_IRUSR | S_IWUSR);  
for (int i = 0; i < 2; ++i){  
    int fd1 = open("ICS.txt", O_RDWR | O_APPEND);  
    int fd2 = open("ICS.txt", O_RDWR);  
    write(fd2, "!!!!!!", 6);  
    write(fd1, "ICS", 3);  
    write(fd, "ics", 3);  
}
```

假设所有系统调用均成功，则这段代码执行结束后，ICS.txt 的内容为：

- A. ICSics
- B. !!!!icsICS
- C. !!!!icsics!!!!ICSICS
- D. !!!!icsICSICS

E2

14. 考虑以下代码，假设 ICS.txt 中的初始内容为"ICS!!!ics!!!"。

```
int fd = open("ICS.txt", O_RDWR | O_CREAT | O_TRUNC,  
S_IRUSR | S_IWUSR);  
for (int i = 0; i < 2; ++i){  
    int fd1 = open("ICS.txt", O_RDWR | O_APPEND);  
    int fd2 = open("ICS.txt", O_RDWR);  
    write(fd2, "!!!!!!", 6);  
    write(fd1, "ICS", 3);  
    write(fd, "ics", 3);  
}
```

假设所有系统调用均成功，则这段代码执行结束后，ICS.txt 的内容为：

- A. ICSics
- B. !!!icsICS
- C. !!!icsics!!!ICSICS
- D. !!!icsICSICS

答案：D（中等）

E3

6. 假设某进程有且仅有五个已打开的文件描述符： $0 \sim 4$ ，分别引用了五个不同的文件，尝试运行以下代码：

```
dup2(3, 2); dup2(0, 3); dup2(1, 10); dup2(10, 4); dup2(4, 0);
```

关于得到的结果，说法正确的是：

- A. 运行正常完成，现在有四个描述符引用同一个文件
- B. 运行正常完成，现在进程共引用四个不同的文件
- C. 由于试图从一个未打开的描述符进行复制，发生错误
- D. 由于试图向一个未打开的描述符进行复制，发生错误

E3

6. 假设某进程有且仅有五个已打开的文件描述符： $0 \sim 4$ ，分别引用了五个不同的文件，尝试运行以下代码：

```
dup2(3, 2); dup2(0, 3); dup2(1, 10); dup2(10, 4); dup2(4, 0);
```

关于得到的结果，说法正确的是：

- A. 运行正常完成，现在有四个描述符引用同一个文件
- B. 运行正常完成，现在进程共引用四个不同的文件
- C. 由于试图从一个未打开的描述符进行复制，发生错误
- D. 由于试图向一个未打开的描述符进行复制，发生错误

答案：A

说明：一开始打开文件描述符(0, 1, 2, 3, 4)对应文件(A, B, C, D, E)，结束后打开描述符(0, 1, 2, 3, 4, 10)对应(B, B, D, A, B, B)，A 正确，B 应该为引用三个不同文件。教材 637 页说明过可以向未打开的描述符进行复制，因此 D 错误；虽然教材未提及从未打开的描述符进行复制的后果，但执行过程中并没有发生这种情况，因此 C 错误。

E4

第四题 (10 分)

分析以下C程序，其中f1.txt和f2.txt为已有用户有读写的文件，初始文件内容为空。

```

1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <sys/types.h>
4. #include <sys/stat.h>
5. #include <fcntl.h>
6. #include <unistd.h>
7. #include <sys/types.h>
8. #include <sys/wait.h>
9.
10. int main()
11. {
12.     int fd1,fd2,fd3,fd4;
13.     int pid;
14.     int c=1;
15.     fd1=open("./f1.txt",O_WRONLY,0);
16.     fd2=open("./f1.txt",O_WRONLY,0);
17.
18.     printf("fd1=%d,fd2=%d;\n",fd1,fd2);
19.
20.     write(fd1,"EECSFKU",7);
21.     write(fd2,"2019",4);
22.
23.     close(fd2);
24.
25.     fd3=open("./f2.txt",O_WRONLY,0);
26.     fd4=dup(fd3);
27.
28.     printf("fd3=%d,fd4=%d;\n",fd3,fd4);
29.
30.     pid=fork();
31.     if(pid==0) {
32.         c--;
33.         write(fd3,"FKU",3);
34.         write(fd4,"ICS",3);
35.         printf("c= %d\n",c);
36.     }
37.     else {
38.         waitpid(-1,NULL,0);
39.         c++;
40.         write(fd3,"2019",4);

```

```

41.     close(fd3);
42.     close(fd4);
43.     printf("c= %d\n",c);
44. }
45.
46. if(c)
47.     write(fd1,"CS",2);
48.     c++;
49.     close(fd1);
50.     printf("c=%d\n",c);
51. }
```

当程序正确运行后，填写输出结果：

- (1) 程序第 18 行：fd1= ①，fd2= ②；
- (2) 程序第 28 行：fd3= ①，fd4= ②；
- (3) 程序第 35、43、50 行输出 c 的值依次分别为：_____；
- (4) 文件 f1.txt 中的内容为：_____；
- (5) 文件 f2.txt 中的内容为：_____。

E4

解答题三

1. 3, 4。注意 0, 1, 2 是标准 I/O 流占用的。
2. 4, 5。因为 fd2 已经被关闭, 其描述符可复用; 另外 fd3, fd4 虽然指针相同, 但是描述符的号码是不同的。
3. 0, 1, 2, 3。父进程等待子进程结束, 所以子进程先输出, 另外父子进程的 count 当然是独立的。
4. 2019PKUCS。对 f1.txt 的写有三处, 其中 20、21 行是独立的, 得到 2019PKU, 最后只有子进程再次写文件, 此时 fd1 的指针在最后, 即 2019PKUCS。
5. PKUICS2019。鉴于 dup 产生 fd4, 而父子进程的文件描述符指向同样的打开文件表条目, 所以对 f2.txt 的写本质上只有一个指针, 因为父进程等待子进程结束, 所以先输出 PKUICS 再输出 2019。

E5

8. 在 Ubuntu Linux 24.04 环境中，关于系统级 IO，下列说法**正确**的是：

- A. 在未进行特殊设置和额外操作的情况下，某进程先调用了 `fprintf()` 函数向某文件写入数据，且返回值恰为传入数据字节数。若向该进程发送 `SIGKILL` 信号终止进程，再打开该文件，一定能看到该进程写入的内容。
- B. 同一进程内的多个线程不加同步地调用 `fprintf()` **会导致**程序崩溃。
- C. 由于慢速系统调用 `read()` 和 `write()` 会潜在地阻塞进程一段较长时间，高级 I/O 函数使用缓冲区来减少对 `read()` 和 `write()` 的调用。
- D. 无论父进程的标准 I/O 缓冲区——例如 `printf()` 函数的缓冲区是否为空，执行 `fork()` 创建的子进程的缓冲区总是为空。

E5

答案：C

A. SIGKILL 会直接终止进程，此时，仍留存在用户态缓冲区的内容尚未被写入磁盘。这里要求“未进行特殊设置和额外操作”保证了缓冲区存在且不被清空。

B. fprintf 是线程安全的，内部有同步机制，不会导致程序崩溃。一个特殊情况是如果两个竞争的 fprintf 调用操作同一个 FILE* 对象，那么文件内容的最终顺序是不确定的。但也不会导致程序崩溃。

C. 正确

D. fork() 创建的子进程的标准 I/O 缓冲区不总是空的，它会复制父进程的标准 I/O 缓冲区的内容。注意——COW 并不会立即复制父进程的缓冲区，而是在子进程或父进程尝试修改缓冲区时才会复制。

E6

答案: C

A. SIGKILL 会直接终止进程, 此时, 仍留存在用户态缓冲区的内容尚未被写入磁盘。这里要求“未进行特殊设置和额外操作”保证了缓冲区存在且不被清空。

B. fprintf 是线程安全的, 内部有同步机制, 不会导致程序崩溃。一个特殊情况是如果两个竞争的 fprintf 调用操作同一个 FILE* 对象, 那么文件内容的最终顺序是不确定的。但也不会导致程序崩溃。

C. 正确

D. fork() 创建的子进程的标准 I/O 缓冲区不总是空的, 它会复制父进程的标准 I/O 缓冲区的内容。注意——COW 并不会立即复制父进程的缓冲区, 而是在子进程或父进程尝试修改缓冲区时才会复制。

E7-1

第四题 (10 分)

Bob 是一名刚刚学完异常的同学,他希望通过配合 kill 和 signal 的使用,能让两个进程向同一个文件中交替地打印出字符。可惜他的 tshlab 做得不过关,导致他写的这个程序有各种 BUG。你能帮帮他吗?

```

1  #include "csapp.h"
2  #define MAXN 6
3  int parentPID = 0;
4  int childPID = 0;
5  int count = 1;
6  int fd1 = 1;
7  void handler1() {
8      if (count > MAXN)
9          return;
10     for (int i = 0; i < count; i++)
11         write(fd1, "+", 1);
12     X
13     kill(parentPID, SIGUSR2);
14 }
15 void handler2() {
16     if (count > MAXN)
17         return;
18     for (int i = 0; i < count; i++)
19         write(fd1, "-", 1);
20     Y
21     kill(childPID, SIGUSR1);
22 }
```

```

24 int main() {
25     signal(SIGUSR1, handler1);
26     signal(SIGUSR2, handler2);
27     parentPID = getpid();
28     childPID = fork();
29     fd1 = open("file.txt", O_RDWR);
```

```

30     if (childPID) {
31         Z
32         kill(childPID, SIGUSR1);
33     }
34     exit(0);
35 }
```

注意: 假设程序能在任意时刻被系统打断、调度,并且调度的时间切片大小是不确定的,可以足够地长。在每次程序执行前, file.txt 是一个已经存在的空文件。

Part A. (1 分) 此时, X 处语句和 Y 处语句都是 count++;, Z 处语句是空语句。Alice 测试该代码,发现有时 file.txt 中没有任何输出! 请解释原因。(提示: 考虑 28 行语句 fork 以后,下一次被调度的进程,并从这个角度回答本题。不需要给出解决方案)

E7-1

第四题 (10 分)

Bob 是一名刚刚学完异常的同学,他希望通过配合 `kill` 和 `signal` 的使用,能让两个进程向同一个文件中交替地打印出字符。可惜他的 `tshlab` 做得不过关,导致他写的这个程序有各种 BUG。你能帮帮他吗?

```

1  #include "csapp.h"
2  #define MAXN 6
3  int parentPID = 0;
4  int childPID = 0;
5  int count = 1;
6  int fd1 = 1;
7  void handler1() {
8      if (count > MAXN)
9          return;
10     for (int i = 0; i < count; i++)
11         write(fd1, "+", 1);
12     X
13     kill(parentPID, SIGUSR2);
14 }
15 void handler2() {
16     if (count > MAXN)
17         return;
18     for (int i = 0; i < count; i++)
19         write(fd1, "-", 1);
20     Y
21     kill(childPID, SIGUSR1);
22 }
```

```

24 int main() {
25     signal(SIGUSR1, handler1);
26     signal(SIGUSR2, handler2);
27     parentPID = getpid();
28     childPID = fork();
29     fd1 = open("file.txt", O_RDWR);
```

```

30     if (childPID) {
31         Z
32         kill(childPID, SIGUSR1);
33     }
34     exit(0);
35 }
```

注意: 假设程序能在任意时刻被系统打断、调度,并且调度的时间切片大小是不确定的,可以足够地长。在每次程序执行前, `file.txt` 是一个已经存在的空文件。

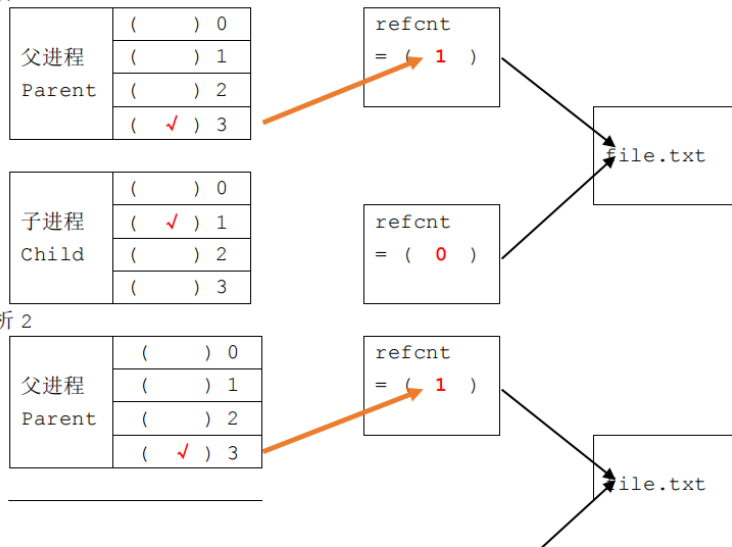
Part A. (1 分) 此时, X 处语句和 Y 处语句都是 `count++`; Z 处语句是空语句。Alice 测试该代码,发现有时 `file.txt` 中没有任何输出! 请解释原因。(提示: 考虑 28 行语句 `fork` 以后,下一次被调度的进程,并从这个角度回答本题。不需要给出解决方案)

【答案】Part A. 如果 28 行 `fork` 执行过后,子进程先被调度了,并且执行完所有代码并退出,那么父进程的 `kill` 操作就无效了。

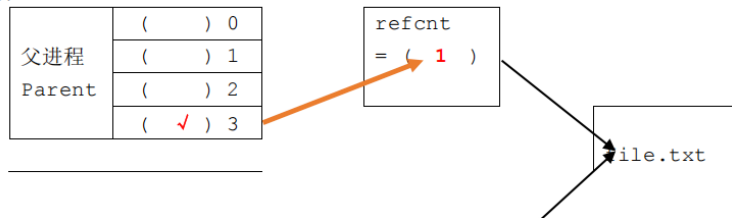
E7-2

Part B.

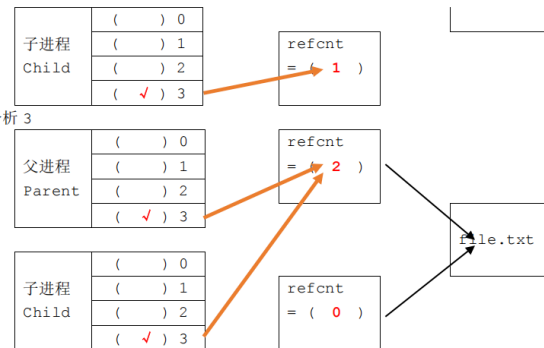
分析 1



分析 2



分析 3



E7-3

Part C. (2 分) Bob 很高兴, 他知道 Part B 的代码是怎么错的了! 不过 Alice 仍然想考考 Bob。对于 Part B 的错误代码, 如果终端上输出的是+++ , 那么

file.txt 中的内容是什么? 请在下框中写出答案。

Part D. (1 分) Bob 修复了 Part B 的问题, 使得代码能够产生预期的输出。现在, Bob 又希望自己的代码最终输出的是+--+--+-----+++++-----, 为此, 他对 X、Y、Z 处做了如下的修改。X、Y 处语句已做如下填写, 请帮助 Bob 补上 Z 处语句。

X 处填写为: `count += 2;`

Y 处填写为: `count += 2;`

Z 处填写为:

E7-3

Part C. (2 分) Bob 很高兴,他知道 Part B 的代码是怎么错的了!不过 Alice 仍然想考考 Bob。对于 Part B 的错误代码,如果终端上输出的是+++,那么

file.txt 中的内容是什么?请在下框中写出答案。

Part D. (1 分) Bob 修复了 Part B 的问题,使得代码能够产生预期的输出。现在,Bob 又希望自己的代码最终输出的是+---+++-+---+---+---+---,为此,他对 X、Y、Z 处做了如下的修改。X、Y 处语句已做如下填写,请帮助 Bob 补上 Z 处语句。

X 处填写为: `count += 2;`

Y 处填写为: `count += 2;`

Z 处填写为:

Part C. `+++++-+++-+---+---+---+---` (6 个+、1 个-、3 个+、2 个-、3 个+、6 个-)

Part D. `count = 2;` (或 `count++;` 等,只要让 count 最终的值是 2 就可以了)

■ 查看答案 ■

E8-1

Part A: 分析下面这段代码的行为, 假设 file.txt 文件存在, 且不需要考虑 open() 函数未能正常打开文件的情况。

```
#include <csapp.h>
int main() {
    int fd[3];
    fd[0] = open("./file.txt", O_RDWR | O_TRUNC, 0);
    fd[1] = open("./file.txt", O_RDWR | O_TRUNC, 0);
    write(fd[0], "Harry", 5);

    if (fork() == 0) {
        dup2(fd[0], fd[1]);
        write(fd[1], "Voldemort", 9);
        fd[2] = open("./file.txt", O_RDWR, 0);
        printf("%d\n", fd[2]);
        exit(0);
    }
    wait(NULL);

    fd[1] = dup(fd[0]);
    write(fd[0], "Hermione", 8);
    write(fd[1], "Ron", 3);

    fd[2] = open("./file.txt", O_RDWR, 0);
    printf("%d\n", fd[2]);
}
```

- 1) 运行这段代码, 在终端上可以看到两行输出, 第一行的内容为_____, 第二行的内容为_____。(每空 1 分)
- 2) 运行这段代码后, file.txt 文件的内容为_____。(3 分)

E8-1

Part A: 分析下面这段代码的行为, 假设 file.txt 文件存在, 且不需要考虑 open() 函数未能正常打开文件的情况。

```
#include <csapp.h>
int main() {
    int fd[3];
    fd[0] = open("./file.txt", O_RDWR | O_TRUNC, 0);
    fd[1] = open("./file.txt", O_RDWR | O_TRUNC, 0);
    write(fd[0], "Harry", 5);

    if (fork() == 0) {
        dup2(fd[0], fd[1]);
        write(fd[1], "Voldemort", 9);
        fd[2] = open("./file.txt", O_RDWR, 0);
        printf("%d\n", fd[2]);
        exit(0);
    }
    wait(NULL);

    fd[1] = dup(fd[0]);
    write(fd[0], "Hermione", 8);
    write(fd[1], "Ron", 3);

    fd[2] = open("./file.txt", O_RDWR, 0);
    printf("%d\n", fd[2]);
}
```

1) 运行这段代码, 在终端上可以看到两行输出, 第一行的内容为_____, 第二行的内容为_____。(每空 1 分)

2) 运行这段代码后, file.txt 文件的内容为_____。

(3 分)

Part A:

1) 5 6

评分标准: 每空 1 分

解析: 本题考察对 dup2() 函数, dup() 函数以及文件标识符分配的理解

第一空: 在 dup2 前已经分配的文件标识符有 fd[0]:3, fd[1]:4, 在 dup2 后文件 fd[1] 仍然为 4, 故新分配的 fd[2] 为 5。

第二空: 在 dup 前已经分配的文件标识符有 fd[0]:3, fd[1]:4, 在 dup 后文件 fd[1] 变为 5, 文件标识符 4 对应的文件仍然没有被关闭, 故新分配的 fd[2] 为 6。

2) HarryVoldemortHermioneRon

评分标准: 本空 3 分, 完全一致才得满分, 可以看出的抄写错误可酌情给 2 分

解析: 本题考察对文件写的理解, 特别是和 fork 函数结合。

不论是 dup, dup2 还是 fork, 都不会创建新的文件, 所以本题中所有的写都是针对同一个文件的写, 因此顺序写入字符串即为答案。

E8-2

Part B: 分析下列代码的行为，其功能是利用信号处理函数输出数列内容。

```
#include <csapp.h>
#define N 10
int flag = 0;
void handler1() {
    for (int i = 0; i < N; i += 2){
        printf("%d ", i)
        fflush(stdout);
    }
```

```
    }
    flag++;
}
void handler2() {
    for (int i = 1; i < N; i += 2){
        printf("%d ", i);
        fflush(stdout);
    }
    flag++;
}
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        signal(SIGUSR1, handler1);
        signal(SIGUSR2, handler2);
        while (flag < 2)
            sleep(1);
    }
    else{
        kill(pid, SIGUSR1);
        kill(pid, SIGUSR2);
        wait(NULL);
    }
    return 0;
}
```

1) 运行这段代码后，发现程序没有输出，且陷入死循环。请简要指出其原因，并给出解决方案。（2 分）

E8-2

Part B: 分析下列代码的行为，其功能是利用信号处理函数输出数列内容。

```
#include <csapp.h>
#define N 10
int flag = 0;
void handler1() {
    for (int i = 0; i < N; i += 2){
        printf("%d ", i)
        fflush(stdout);
    }
}
```

Part B:

1) 问题：子进程信号注册函数位置存在问题。父进程在 kill 后，子进程的信号

18

处理函数可能未完全注册，导致信号丢失。解决方案：将 `signal(SIGUSR1, handler1); signal(SIGUSR2, handler2);` 两行提至 `fork` 前。

评分标准：原因 1 分，解决方案 1 分。意思对即可。

```
    }
    flag++;
}
void handler2() {
    for (int i = 1; i < N; i += 2){
        printf("%d ", i);
        fflush(stdout);
    }
    flag++;
}
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        signal(SIGUSR1, handler1);
        signal(SIGUSR2, handler2);
        while (flag < 2)
            sleep(1);
    }
    else{
        kill(pid, SIGUSR1);
        kill(pid, SIGUSR2);
        wait(NULL);
    }
    return 0;
}
```

1) 运行这段代码后，发现程序没有输出，且陷入死循环。请简要指出其原因，并给出解决方案。（2 分）

E8-3

2) 在解决上述问题之后，第一次运行得到“1357902468”；但再次运行得到的却是“1302468579”，与期望不符。请结合“编写信号处理函数”的知识，补全 handler1() 函数中缺失的代码，以保证运行结果的正确性。(3 分)

```
void handler1(){
    sigset_t mask, prev_mask;
    _____ A _____ ( _____ B _____ );
    _____ C _____ ( _____ D _____ , &mask, &prev_mask);
    for (int i = 0; i < N; i += 2){
        printf("%d ", i)
        fflush(stdout);
    }
    flag++;
    _____ E _____ ( _____ F _____ , &prev_mask);
}
```

基于下列选项选择序号填写：

E8-3

2) 在解决上述问题之后，第一次运行得到“1357902468”；但再次运行得到的却是“1302468579”，与期望不符。请结合“编写信号处理函数”的知识，补全 handler1() 函数中缺失的代码，以保证运行结果的正确性。(3 分)

```
void handler1(){
    sigset_t mask, prev_mask;
    _____ A _____ ( _____ B _____ );
    _____ C _____ ( _____ D _____ , &mask, &prev_mask);
    for (int i = 0; i < N; i += 2){
        printf("%d ", i)
        fflush(stdout);
    }
    flag++;
    _____ E _____ ( _____ F _____ , &prev_mask);
}
```

基于下列选项选择序号填写：

2) ① ⑦ ② ④ ② ⑥

评分标准：两个一句，每句 1 分，全对得 1 分，否则不得分。

E8-4

Part C: 分析下面这段代码的行为，假设 `file.txt` 文件存在且初始内容为空，并且不需要考虑 `open()` 函数未能正常打开文件的情况。

```
#include <csapp.h>

int main() {
    char *numbers = "0123456789";

    char buffer[100];
    buffer[0] = '\0';

    pid_t father_pid = getpid();

    for (int i = 0; i < 10; ++i) {
        if (fork() != 0) {
            strncat(buffer, numbers + i, 1);
        }
    }

    int fd = open("./file.txt", O_RDWR, 0);
    write(fd, buffer, strlen(buffer));

    while(wait(NULL) != -1) {}

    if (getpid() == father_pid) {
        close(fd);
        fd = open("./file.txt", O_RDONLY, 0);
        char content[100];
        read(fd, content, 100);
        printf("%c\n", content[3]);
        printf("%c\n", content[6]);
    }
    exit(0);
}
```

如果你不清楚 `string.h` 中 `strncat()` 函数的行为，可以参考下列说明：

`strncat()` 函数用于将一个字符串追加到另一个字符串的末尾。它接受一个目标字符串 `dest`，一个源字符串 `src` 和最大追加的字符数 `n`。在 `src` 的长度至少为 `n` 时，这个函数将 `src` 的前 `n` 个字符拼接到 `dest` 字符串的后面。例如：

```
char *dest = "Astarion";
char *src = "Shadowheart";
strncat(dest, src, 6);
printf("%s\n", dest);
```

这段代码将输出 `AstarionShadow`。

- 1) 运行上述代码，在终端上会看到两行输出，第一行的内容可能为_____，第二行的内容可能为_____。（你需要列出所有可能出现的输出，每种可能性用逗号隔开，每空 1 分）
- 2) 在运行这段代码之后，`file.txt` 中的内容有_____种可能的结果。（3 分）（注意，字符串级别相等的内容属于同一种可能的结果）

E8-4

Part C:

5) 3, 4, 5, 6, 7, 8, 9 6, 7, 8, 9

评分标准：每空 1 分，完全一致才得分

解析：本题考察对于父子进程局部变量关系，以及并发的理解。

观察循环体，可以发现对于最终形成的每一个进程，都经历了十次选择，也即：

for i in 0..10, 是否将 i 加到 buffer 的最后

并且每一个进程至少有一个选择是不同的，这导致最终形成的每一个进程的内容都不一样，且共形成了 2^{10} 个进程。（如果不能理解这个过程，想想你在 AI 课上学到的决策树，你可以完全类似地画一棵进程树出来）

下面先考虑 content[9]可能出现的内容，已知必须写入一个长度至少为 10 的 buffer 才能让 content[9]出现字符，而所有进程的 buffer 中，只有总父亲进程的 buffer="0123456789"满足长度要求，故 content[9]只能为 9。

接下来考虑 content[j], j in 0..10, 我们需要写入一个长度至少为 j + 1 的 buffer 才能让 content[j]的字符发生改变，而第 buffer[j]之前的 j 个数字又必须比 buffer[j]的数字小，这就意味着 buffer[j]至少为 j。在 10 次选择中，我们至多选择 9 - j 次不将数字 i 加入 buffer 最后，我们容易发现全选靠前的数字不加入可以得到最大的 content[j]，易得最大的 content[j]为 9，同样的方法可以发现 j..10 均能取到。

这样本题的答案就被解出来了。

6) 10!

评分标准：本空 3 分，答案为 10 的阶乘，写成 3628800 或其他等价表示也能得分。

解析：根据上一问的结果，我们可以推出 content[i]有 $10-i$ 种取法，那么根据乘法原理，content 最多有 $10!$ 种可能性。

下面证明确实可以取到 $10!$ 种可能性，对于任意的一种可能性：

$x_0 \ x_1 \ x_2 \ \cdots \ x_9$

我们可以这样构造进程写的执行序列来得到它：

1. 遍历 i, i 从 9 取到 0, 每次递减 1

2. 对于每一个 i, 寻找一个进程，这个进程的 buffer 长度恰好为 i + 1, 且最后一位数字为 x_i 。由上一问的论证，这样的进程恒存在。

3. 遍历结束，我们得到了十个不同的进程，我们先以任意顺序完成这十个进程之外的进程的写，然后再依次执行这十个进程的写，这样我们就构造出了我们想要的序列。

Notice

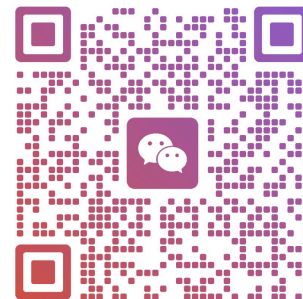
Shell Lab 明晚截止

THANKS

Made by WEB-05

webrun@stu.pku.edu.cn

Reference: [WalkerCH]'s and [Arthals]'s presentations.



扫一扫上面的二维码图案，加我为朋友。